

ANDROID STUDIO

KOTLIN- BALIO NULUAK ETA BALIO NULUEN SEGURTASUNA

HELBURUAK

Gai honetan kontzeptu hauek landuko ditugu:

1. Kotlin-eko balio nuluen hastapenak
2. Oraingo balioen eta ez daudenen arteko aldea
3. NULL balioak onartzen dituzten moten eta NULL balioak onartzen ez dituzten moten arteko aldea
4. Nola erlazionatzen dira NULL balioak onartzen dituzten motak eta NULL balioak onartzen ez dituztenak?
5. Nola erabili baldintzak balio nuluak egiaztatzeko

KOTLIN-EKO BALIO NULUEN HASTAPENAK

Balore nuluen kontzeptuak aztertzeko adibide baten bitartez egingo dugu.

Jamesek kafe postu bat jarri du hiriaren erdigunean eta prest dago bere edari bikaina partekatzeko. Katilu bakoitza banatu ondoren, kafearen aipamen bat uzteko eskatzen die bezeroei, interesa izan dezaketen beste pertsona batzuekin kalifikazioak partekatu ahal izateko.

Kotlin programa batean bitartez kalifikazioen jarraipena egin nahi duenez, klase erraz hau idatzi du:

```
class CoffeeReview(  
    val name: String,  
    val comment: String,  
    val stars: Int  
)
```

KOTLIN-EKO BALIO NULUEN HASTAPENAK

Eguneko lehen hiru gonbidatuek bete dituzte iritzi-txartelak:

The image shows three hand-drawn review cards for 'JAMES' COFFEE'. Each card has a blue header with the text 'JAMES' COFFEE'. Below the header, there are three sections: 'NAME:', 'COMMENT:', and 'STARS:'. The first card is filled out by Sara, who gave 5 stars and wrote 'Loved the coffee!'. The second card is filled out by Toby, who gave 4 stars and wrote 'PRETTY GOOD!'. The third card is filled out by Lucy, who gave 5 stars and wrote 'Will buy this again!'. The stars are represented by a row of numbers from 0 to 5, with the chosen star circled in red.

NAME:	COMMENT:	STARS:
Sara	Loved the coffee!	0 1 2 3 4 5
TOBY	PRETTY GOOD!	0 1 2 3 4 5
Lucy	Will buy this again!	0 1 2 3 4 5

KOTLIN-EKO BALIO NULUEN HASTAPENAK

Jamesek CoffeeReview objektu batzuk sortzen ditu jasotako hiru aipamenak erregistratzeko.

```
val saraReview = CoffeeReview("Sara", "Loved the coffee!", 5)  
val tobyReview = CoffeeReview("Toby", "Pretty good!", 4)
```

Baina Lucy-n aipamena erregistratzerakoan konturatzen da ez duela izarrik jarri. Beraz 0 bezala jartzea erabakitzen du.

```
val lucyReview = CoffeeReview("Lucy", "Will buy this again!", 0)
```

KOTLIN-EKO BALIO NULUEN HASTAPENAK

Jada James prest dago aipamenak pantaila batean erakusteko, eta, beraz, funtzio sinple bat idatziko du:

```
fun printReview(review: CoffeeReview) =  
    println("${review.name} gave it ${review.stars} stars!")  
  
println("Latest coffee reviews")  
println("-----")  
printReview(saraReview)  
printReview(tobyReview)  
printReview(lucyReview)
```

```
Últimas reseñas de café  
-----  
¡Sara le dio 5 estrellas!  
¡Toby le dio 4 estrellas!  
¡Lucy le dio 0 estrellas!
```

KOTLIN-EKO BALIO NULUEN HASTAPENAK

Jada James prest dago aipamenak pantaila batean erakusteko, eta, beraz, funtzio sinple bat idatziko du:

```
fun printReview(review: CoffeeReview) =  
    println("${review.name} gave it ${review.stars} stars!")  
  
println("Latest coffee reviews")  
println("-----")  
printReview(saraReview)  
printReview(tobyReview)  
printReview(lucyReview)
```

```
Últimas reseñas de café  
-----  
¡Sara le dio 5 estrellas!  
¡Toby le dio 4 estrellas!  
¡Lucy le dio 0 estrellas!
```

Bere soluzioak ondo funtzionatuko zuela pentsatu zuen, baina beste gonbidatuek aipamenak ikusten dutenean, pentsatuko dute: "Hara, Lucyri kafea gustatu ez bazaio, agian ez da ona izango. Beste nonbaitera joango naiz".

KOTLIN-EKO BALIO NULUEN HASTAPENAK

James konturatu zen zero izarreko kalifikazioa eta kalifikaziorik ez izatea, ez dela gauza bera.

The diagram illustrates two different ways to represent a zero-star rating on a review form for 'JAMES' COFFEE'.

Left Form (Zero-Star Rating):

- NAME: Mike
- COMMENT: Sooo bad!!
- STARS: 0 1 2 3 4 5 (The '0' is circled)
- Annotation: An arrow points from the text 'Zero-Star Rating' to the circled '0'.

Right Form (No Star Rating):

- NAME: Lucy
- COMMENT: Will buy this again!
- STARS: 0 1 2 3 4 5 (No star is selected)
- Annotation: An arrow points from the text 'No Star Rating' to the stars section.

KOTLIN-EKO BALIO NULUEN HASTAPENAK

James konturatu zen zero izarreko kalifikazioa eta kalifikaziorik ez izatea, ez dela gauza bera.



Norbaitek kalifikaziorik uzten ez duenean, Jamesek ez du pantailan zero izar kalifikazioa erakutsi nahi. Kotlin-i izarren kalifikaziorik ez zuela utzi esateko modu bat behar du. Nola egin dezakezu hori?

ORAINGO BALIOEN ETA EZ DAUDENEN ARTEKO ALDEA

Orain arte idatzi dugun kode osoan, balio bat duten aldagaiak sortu ditugu. Beste era batera esanda, beti izan dugu zerbait edukiontzi horren barruan. Adibidez, stars edukiontzi batek Int bat dauka, 5 zenbakiarekin.



ORAINGO BALIOEN ETA EZ DAUDENEN ARTEKO ALDEA

CoffeeReview klaserako, kontainer stars bat behar dugu, balio bat izan dezakeena edo ez. Gonbidatuak kalifikazio bat uzten duenean, edukiontziak balio hori izan behar du, baina kalifikatzea ahazten bazaio, edukiontziak hutsik egon behar du. Balio bat jartzeko edo hutsik uzteko edukiontzi bat nahi dugu. Horrek bi termino berri sortzen ditu:

- ▶ Aldagai batek bere barruan balio bat duenean, **balioa bertan dagoela (presente)** esango dugu.
- ▶ Aldagai batek barruan baliorik ez duenean, **balioa falta dela (ausente)** esango dugu.

ORAINGO BALIOEN ETA EZ DAUDENEN ARTEKO ALDEA



ORAINGO BALIOEN ETA EZ DAUDENEN ARTEKO ALDEA

Kotlinek null izeneko gako-hitza erabiltzen du baliorik ez dagoela adierazteko. Null bidez esleitzen den aldagai bat hutsik dagoen kuboaren bezalakoa da. Kalifikaziorik ez duen aipamen baterako, null-ean ezarri nahi dugu. Saia gaitezke null bat esleitzen Lucy CoffeeReview-ena eraikitzen dugunean, baina egiten dugunean, errore bat lortzen dugu:

```
val lucyReview = CoffeeReview("Lucy", "Will buy this again!", null)
```

Kotlin-ek ez digu utziko edozein aldagairi null balioak esleitzen. Horren ordean, argi eta garbi adierazi behar duzu aldagai bat noiz egon daitekeen hutsik.

NULL BALIOAK ONARTZEN DITUZTEN MOTEN ETA NULL BALIOAK ONARTZEN EZ DITUZTEN MOTEN ARTEKO ALDEA

Kotlin-en, hainbat mota erabiltzen ditugu aldagai bat null-en ezar daitekeen ala ez adierazteko. NULL balioak onartzen dituzten motek galdera-ikur batekin amaitzen dute.

<u>NON-NULLABLE</u>	<u>NULLABLE TYPES</u>
String	String?
Int	Int?
Boolean	Boolean?
Circle	Circle?
SchnauzerBreed	SchnauzerBreed?

NULL BALIOAK ONARTZEN DITUZTEN MOTEN ETA NULL BALIOAK ONARTZEN EZ DITUZTEN MOTEN ARTEKO ALDEA

Kotlin-en, hainbat mota erabiltzen ditugu aldagai bat null-en ezar daitekeen ala ez adierazteko. NULL balioak onartzen dituzten motek galdera-ikur batekin amaitzen dute.

<u>NON-NULLABLE</u>	<u>NULLABLE TYPES</u>
String	String?
Int	Int?
Boolean	Boolean?
Circle	Circle?
SchnauzerBreed	SchnauzerBreed?

NULL BALIOAK ONARTZEN DITUZTEN MOTEN ETA NULL BALIOAK ONARTZEN EZ DITUZTEN MOTEN ARTEKO ALDEA

```
val stars: Int = null
```

```
val stars: Int? = null
```

Orain, stars null gisa konfigura daiteke. Jakina, balio oso normal gisa ere konfigura daiteke.

```
val saraStars: Int? = 5  
val tobyStars: Int? = 4  
val lucyStars: Int? = null
```

NOLA ERLAZIONATZEN DIRA NULL BALIOAK ONARTZEN DITUZTEN MOTAK ETA NULL BALIOAK ONARTZEN EZ DITUZTENAK?

Nahiz eta Int eta Int? erlazionatuta egon, bi mota desberdin izaten jarraitzen dute eta ezin dugu Int? bat erabili Int bat erabiliko genukeen edozein lekutan.

```
fun printReview(name: String, stars: Int) =  
    println("$name gave it $stars stars!")  
  
val saraStars: Int? = 5  
  
printReview("Sara", saraStars)
```

NOLA ERLAZIONATZEN DIRA NULL BALIOAK ONARTZEN DITUZTEN MOTAK ETA NULL BALIOAK ONARTZEN EZ DITUZTENAK?

NULL balioak onartuko dituen tipo baten zain

James sortutako klaseari funtzio berri bat gehituko diogu:

```
fun orderCoffee(payment: Payment?): Coffee {  
    return Coffee()  
}
```

Jakina, norbaitek kafea eskatu eta ordaintzen badu, Jamesek gustura aterako dio kafea!



NOLA ERLAZIONATZEN DIRA NULL BALIOAK ONARTZEN DITUZTEN MOTAK ETA NULL BALIOAK ONARTZEN EZ DITUZTENAK?

Payment argumentu bat orderCoffee () funtziora pasatzea eszenatoki hau bezalakoa da: gonbidatua ordainketa egiten ari da behin betiko.

```
val payment: Payment = Payment()  
val coffee = orderCoffee(payment)
```

Orain, imajinatu norbait sartzen dela eta esaten duela: "Kutxa honek dirua izan dezake edo hutsik egon daiteke. Barruan dagoenarekin gera zaitezke ". Jamesek dio: "Hutsik badago ere, ez dago arazorik. Azken finean, ongintzazko erakundea gara. Hemen duzu kafea! ".



NOLA ERLAZIONATZEN DIRA NULL BALIOAK ONARTZEN DITUZTEN MOTAK ETA NULL BALIOAK ONARTZEN EZ DITUZTENAK?

Payment? argumentua orderCoffee () funtzioari pasatzen diogunean, gonbidatua James-i kutxa misteriotsu bat ematen ari balitzaio bezala da, dirua edo ezer ez duena.

```
val payment: Payment? = Payment() // or you could set this to null  
val coffee = orderCoffee(payment)
```

Laburbilduz, NULL balioak onartzen duen parametroa (Payment?) duen funtzio bat ongintzako erakunde bat bezalakoa da: NULL balioak onartzen ez dituen mota bat duen argumentu bat onar dezake (Payment bezala), eta NULL balioak onartzen dituen mota bat duen argumentu bat ere onar dezake (Payment? bezala). Bi aukeretako ondo egongo lirateke.

NOLA ERLAZIONATZEN DIRA NULL BALIOAK ONARTZEN DITUZTEN MOTAK ETA NULL BALIOAK ONARTZEN EZ DITUZTENAK?

```
val payment: Payment = Payment()  
orderCoffee(payment)
```



```
val payment: Payment? = Payment()  
orderCoffee(payment)
```



```
fun orderCoffee(payment: Payment?): Coffee {  
    return Coffee()  
}
```

NOLA ERLAZIONATZEN DIRA NULL BALIOAK ONARTZEN DITUZTEN MOTAK ETA NULL BALIOAK ONARTZEN EZ DITUZTENAK?

Funtzioa honela definituko bagenu:

```
fun orderCoffee(payment: Payment): Coffee {  
    return Coffee()  
}
```

```
val payment: Payment = Payment()  
orderCoffee(payment)
```



```
val payment: Payment? = Payment()  
orderCoffee(payment)
```



```
fun orderCoffee(payment: Payment): Coffee {  
    return Coffee()  
}
```

NOLA ERABILI BALDINTZAK BALIO NULUAK EGIAZTATZEKO

Hona hemen kafetegiko negozioaren funtzioa:

```
fun orderCoffee(payment: Payment): Coffee {  
    return Coffee()  
}
```

Bezera hutsik egon zitekeen kutxa batekin ordaintzen saiatu zenean, hau gertatu zen:

```
val payment: Payment? = Payment()  
val coffee = orderCoffee(payment)
```

Hau konpontzeko era bat:

```
val payment: Payment? = Payment()  
  
if (payment != null) {  
    val coffee = orderCoffee(payment)  
} else {  
    println("I can't order coffee today")  
}
```

NOLA ERABILI BALDINTZAK BALIO NULUAK EGIAZTATZEKO

Horri **bihurketa adimenduna** deitzen zaio.

```
val payment: Payment? = Payment()
```

Type of payment is Payment? here (nullable)

```
if (payment != null) {
```

```
    val coffee = orderCoffee(payment)
```

Type of payment is Payment here (non-nullable)

```
} else {
```

```
    println("I can't order coffee today")
```

Type of payment is back to Payment? again here

```
}
```

NOLA ERABILI BALDINTZAK BALIO NULUAK EGIAZTATZEKO

Jaurtiketa adimendunek when baldintzapeko batekin ere funtzionatzen dute, hala nola:

```
when (payment) {  
    null -> println("I can't order coffee today")  
    else -> orderCoffee(payment)  
}
```